

Interrupt-Driven PWM Motor Control on AVR

I. Introduction

The objective of this project is to implement a Pulse Width Modulation (PWM)-based speed control for a DC motor using the ATmega324PB. This requires generating discrete duty-cycle levels through digital I/O, measuring rotational speed of an attached DC motor using an opto-interrupter and external interrupts, and presenting real-time operating data on an I2C OLED display.

PWM works by rapidly switching the motor's voltage on and off. At sufficient speed, the motor coils can't respond to individual transitions and, instead, to the average voltage over time. Thus, the longer the voltage stays on during each cycle, the faster the motor spins. For example, a duty cycle of 50% will allow the motor to receive roughly half the full supply voltage. The ATmega324PB has built-in timers to handle voltage switching automatically, so it doesn't require constant CPU supervision.

To determine how fast the motor is spinning, an opto-interrupter is used. The device has a phototransistor which sends a pulse every time one of the gaps in the spinner passes in front of the LED. As there are two gaps on opposite sides of the spinner, each pulse represents half of a rotation. By counting how many pulses happen in a time interval, we can calculate the motor's speed in revolutions per minute (RPM).

External interrupts simplify this process, as the processor automatically runs whenever a pulse is sent without us having to keep checking the sensor. Combining the PWM for speed control, interrupts for counting pulses, and Inter-Integrated Circuit (I2C) display for displaying the performance, we have a system that can control the motor and display its speed in real time.

II. Procedure

a. Flowchart

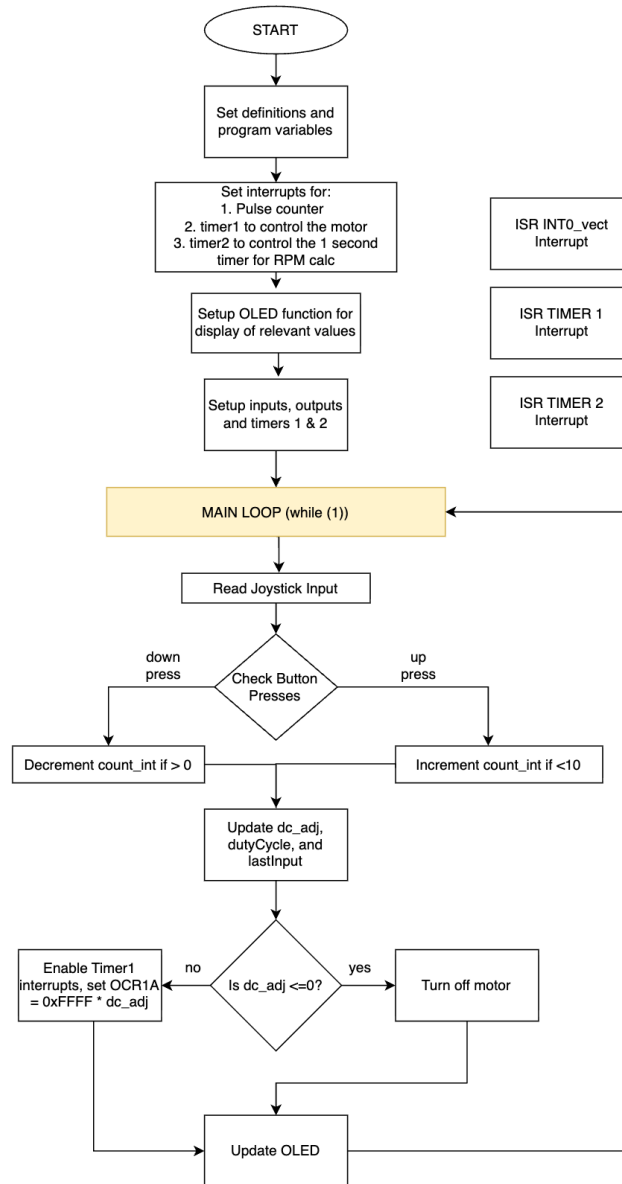


Figure 1: Flowchart of the Program Logic

b. Method:

The first three blocks of code are pure initialization, providing access to AVR I/O registers, enabling use of interrupts and libraries for I2C communication, and defining and declaring variables.

The Interrupt Service Routine (ISR) section defines the special functions that trigger asynchronously of the main program, allowing the microcontroller to handle events as they occur. ISR(INT0_vect) increments the pulse counter whenever the opto-interrupter triggers.

Timer1 compare interrupts are used to manually toggle the motor control pin, producing a PWM waveform by setting the pin high at OCR1B=0 and low at OCR1A. The ISR for Timer2 counts overflows (when it wraps from 255 to 0) to allow for a 1-second interval timer that is involved in the RPM calculation.

The OLED_display function is a straight-forward code block that displays mode (duty cycle divided by 10), PWM% or duty cycle, RPM, and a running total of the interrupt count (which overflows and resets at 65535).

Within the main program, the first code block is another initialization block, which sets the I2C display and relevant input and output pins. The second block configures INT0 so that it detects the opto-interrupter using a falling-edge trigger. The remaining two blocks initialize Timer1 and Timer2, respectively.

Timer1 provides timing of the PWM by triggering compare interrupts. It is set to CTC mode with ICR1 as TOP (Mode 12, i.e. WGM=1100) with no pre-scaler. In this mode, the timer counts from 0 up to ICR1, then resets. ICR1 thus defines the full PWM period as follows:

$$T_{period} = (ICR1 + 1) * \frac{Prescaler}{f_{clk}} = (65535 + 1) * \frac{1}{16000000} = 4.096 \text{ ms}$$

The motor duty cycle is determined by OCR1A relative to ICR1:

$$OCR1A = ICR1 * dc_adj$$

When TCNT1 reaches OCR1A, the COMPA ISR triggers and turns the motor off. The motor is then turned on when TCNT1 matches OCR1B, which in this case is set to 0. This triggers the COMPB ISR, turning the motor on at the beginning of each timer cycle. This combination of ISRs produces a software-controlled PWM signal whose period is set by ICR1, scaled by dc_adj.

Timer2 is used to measure approximately 1-second intervals for RPM calculation. It is set to normal mode with a 1024 pre-scaler. In this mode, the 8-bit timer counts from 0 to 255, giving an overflow interval of:

$$T_{period} = (256 - T_{preload}) * \frac{Prescaler}{f_{clk}} = (256 - 0) * \frac{1024}{16000000} = 16.384 \text{ ms}$$

Each overflow triggers the Timer2 overflow ISR, which increments a counter. Counting 61 yields approximately 1 second: $T_{1s} = 16.384 \text{ ms} * 61 = 0.999424 \text{ s}$.

In the while loop, the joystick is continually monitored for up and down button presses. So long as the mode is between 0-10, this updates the 'dc_adj' variable, which scales the duty cycle interrupt, as well as updates the I2C display. If dc_adj = 0, the motor is shut off entirely. The code uses 'lastInput' to detect changes rather than raw button level, preventing repeated triggers while a button is held.

c. Hardware Design:

The hardware system consists of the following components:

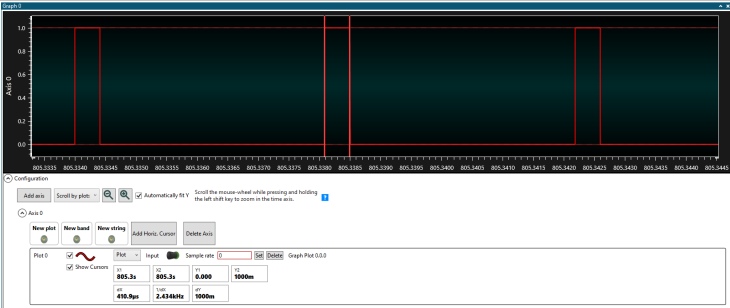
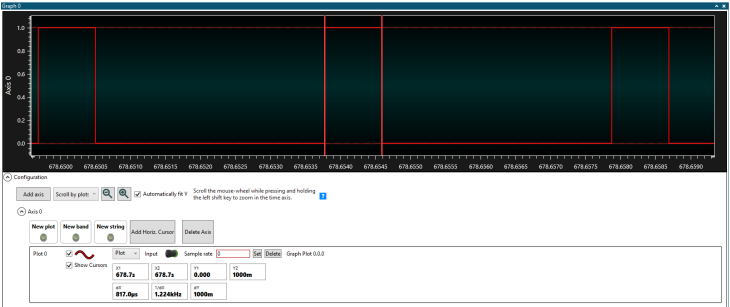
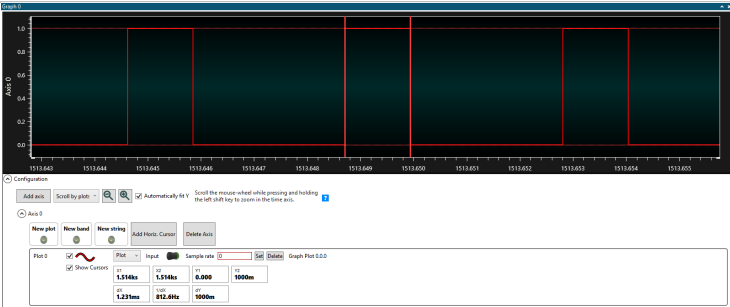
1. **ATmega324PB Microcontroller:** The central controller which generates PWM signals using Timer1, handles external interrupts (INT0) from the opto-interrupter, and communicates with the OLED display over I²C. All control logic, duty cycle adjustments, and RPM calculations are executed on this microcontroller unit (MCU). It operates with a 16 MHz internal clock.
 - a. **Joystick (PORTB)** – Integrated directly onto the MCU, the joystick allows for user adjustment of the PWM duty cycle. Up and down buttons increment or decrement the duty cycle in 10% steps and pull-up resistors ensure defined logic levels when buttons are unpressed.
 - b. **Timers:**
 - i. Timer1 (16-bit) – This timer generates PWM signals for the motor with OCR1A controlling on-time and OCR1B controlling start of cycle. The period is 4.096 ms (ICR1=65535, no prescaler).
 - ii. Timer2 (8-bit) – This timer measures 1-second intervals for RPM calculation using overflows (pre-scaler 1024).
2. **DC Motor:** The actuator whose speed is controlled via the PWM. Its shaft is fitted with a spinner containing slotted sections to generate pulses for speed measurement.
3. **Opto-Isolator:** The opto-isolator interfaces the MCU with the motor power circuit. It separates the 9V that power the motor from the 3.3V MCU logic. A PNP Darlington transistor amplifies the opto-isolator output to drive the motor.
4. **Opto-Interrupter Sensor:** The sensor is mounted so that the spinner passes through its slot. Each opening in the spinner generates a voltage change on PD2, turning the LED off, triggering INT0 and incrementing a pulse counter. Two slots per revolution are used to compute RPM.
5. **I²C OLED Display:** The OLED displays real-time data including mode (1–10), PWM percentage, RPM, and interrupt counts.
6. **Power Supply:** A 9V battery powers the motor through the opto-isolator. The MCU and OLED receive regulated 3.3V or 5V as appropriate from the expansion board.

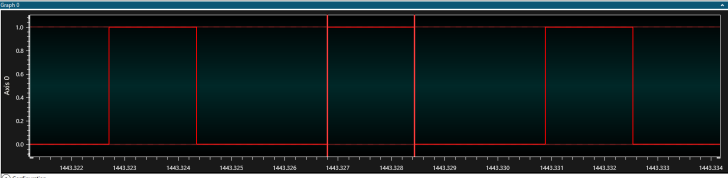
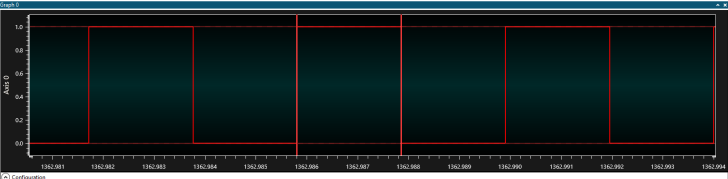
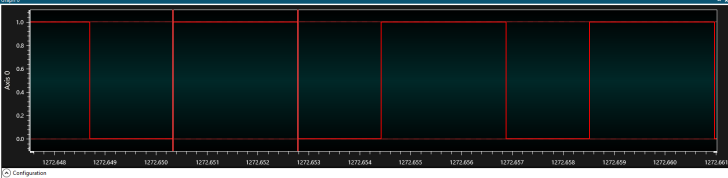
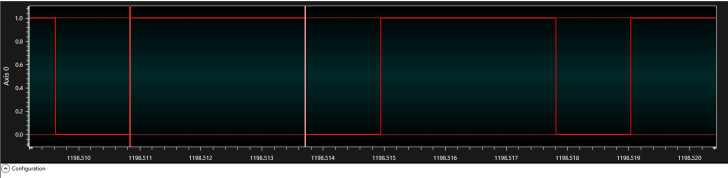
III. Results

a. Result 1

This section demonstrates how the joystick commands translate into actual PWM duty cycles observed on the microcontroller output. Using the Data Visualizer, each joystick step from 0-100% is measured and plotted to show how the control input maps to the generated PWM signal.

Table 1. PWM Table

PWM Counter Value	PWM Duty Cycle (%)	On-Time Pulse Length (ms) / Time Period, T (ms)	Image of Motor PWM (Data Visualizer Screenshot)
1	10	0.4096 ms / 4.096 ms	 ON Signal, $dx = 0.4109$ ms
2	20	0.8192 ms / 4.096 ms	 ON Signal, $dx = 0.8170$ ms
3	30	1.229 ms / 4.096 ms	 ON Signal, $dx = 1.231$ ms

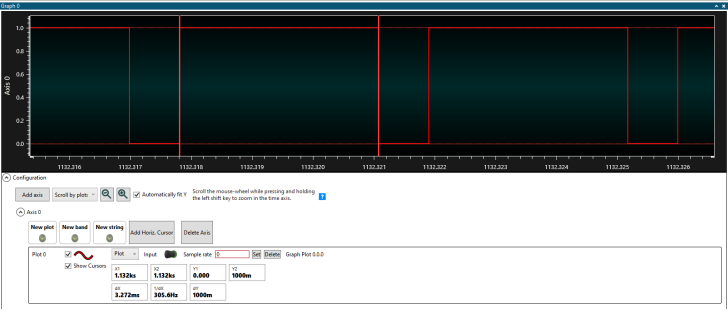
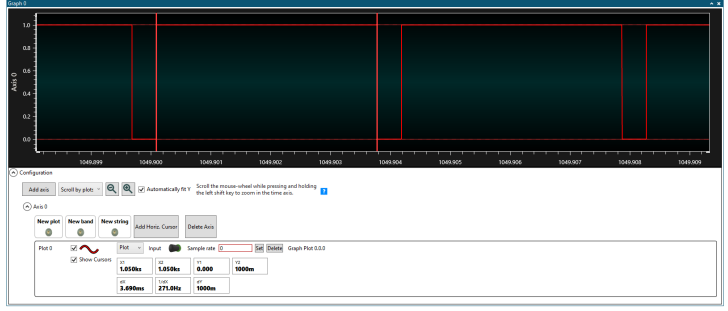
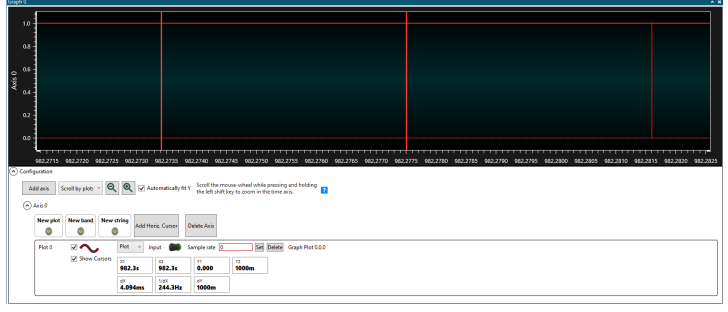
4	40	1.638 ms / 4.096 ms	 Configuration Axis 0 New plot Plot 0 <table><tr><td>1.443ms</td><td>1.443ms</td><td>0.000</td><td>1000m</td></tr><tr><td>1.638ms</td><td>811.45Hz</td><td>1000m</td><td></td></tr></table>	1.443ms	1.443ms	0.000	1000m	1.638ms	811.45Hz	1000m	
1.443ms	1.443ms	0.000	1000m								
1.638ms	811.45Hz	1000m									
5	50	2.048 ms / 4.096 ms	 Configuration Axis 0 New plot Plot 0 <table><tr><td>1.383ms</td><td>1.383ms</td><td>0.000</td><td>1000m</td></tr><tr><td>2.053ms</td><td>487.0Hz</td><td>1000m</td><td></td></tr></table>	1.383ms	1.383ms	0.000	1000m	2.053ms	487.0Hz	1000m	
1.383ms	1.383ms	0.000	1000m								
2.053ms	487.0Hz	1000m									
6	60	2.458 ms / 4.096 ms	 Configuration Axis 0 New plot Plot 0 <table><tr><td>1.273ms</td><td>1.273ms</td><td>0.000</td><td>1000m</td></tr><tr><td>2.456ms</td><td>407.2Hz</td><td>1000m</td><td></td></tr></table>	1.273ms	1.273ms	0.000	1000m	2.456ms	407.2Hz	1000m	
1.273ms	1.273ms	0.000	1000m								
2.456ms	407.2Hz	1000m									
7	70	2.867 ms / 4.096 ms	 Configuration Axis 0 New plot Plot 0 <table><tr><td>1.199ms</td><td>1.199ms</td><td>0.000</td><td>1000m</td></tr><tr><td>2.869ms</td><td>348.8Hz</td><td>1000m</td><td></td></tr></table>	1.199ms	1.199ms	0.000	1000m	2.869ms	348.8Hz	1000m	
1.199ms	1.199ms	0.000	1000m								
2.869ms	348.8Hz	1000m									

ON Signal, $dx = 1.635$ ms

ON Signal, $dx = 2.053$ ms

ON Signal, $dx = 2.456$ ms

ON Signal, $dx = 2.869$ ms

8	80	3.2768 ms / 4.096 ms	 ON Signal, $dx = 3.272$ ms
9	90	3.686 ms / 4.096 ms	 ON Signal, $dx = 3.690$ ms
10	100	4.096 ms / 4.096 ms	 ON Signal, $dx = 4.094$ ms

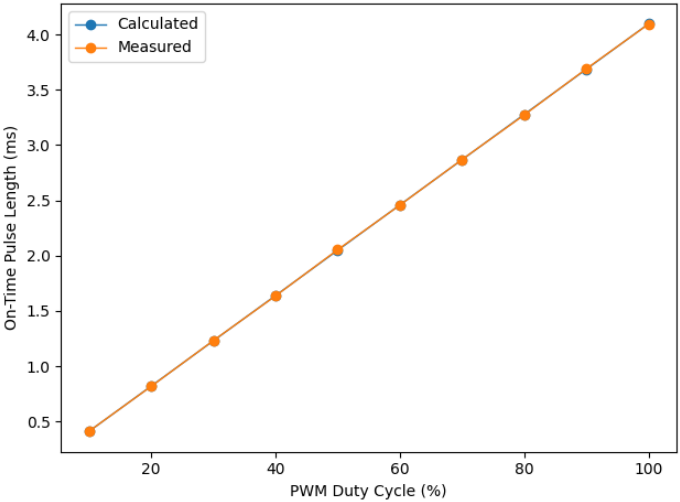


Figure 2: Duty Cycle vs On-Time Pulse Length: Calculated vs Measured

Explanation of result: The measured-ON signal for the PWM tracks the calculated values with sub-percent deviation across all duty cycles. Errors are small and exhibit no systematic drift, indicating stable timing behavior and accurate PWM generation. The result verifies that the motor is receiving the proper signal to scale its power output. Both upward and downward sweeps from 0% to 100% and back were verified on the Data Visualizer in the accompanying demonstration video, confirming correct response to joystick input in both directions.

b. Result 2

This section evaluates motor speed using the optical sensor inputs. Pulse counts from the interrupter are recorded for each duty cycle and converted to RPM. The resulting graphs show how RPM and effective DC voltage vary with duty cycle. Note that the DC Voltage is calculated, not measured. Representative OLED readings at selected operating points are included to confirm the measured values.

Table 2. Summary Table

Joystick Up-Counter	PWM Duty Cycle (%)	DC Voltage (Calculated) [V]	RPM
0	0	0.00	0
1	10	0.90	2880
2	20	1.80	6540
3	30	2.70	9120
4	40	3.60	10800
5	50	4.50	12540
6	60	5.40	13620
7	70	6.30	14700
8	80	7.20	15240
9	90	8.10	15720
10	100	9.00	16680

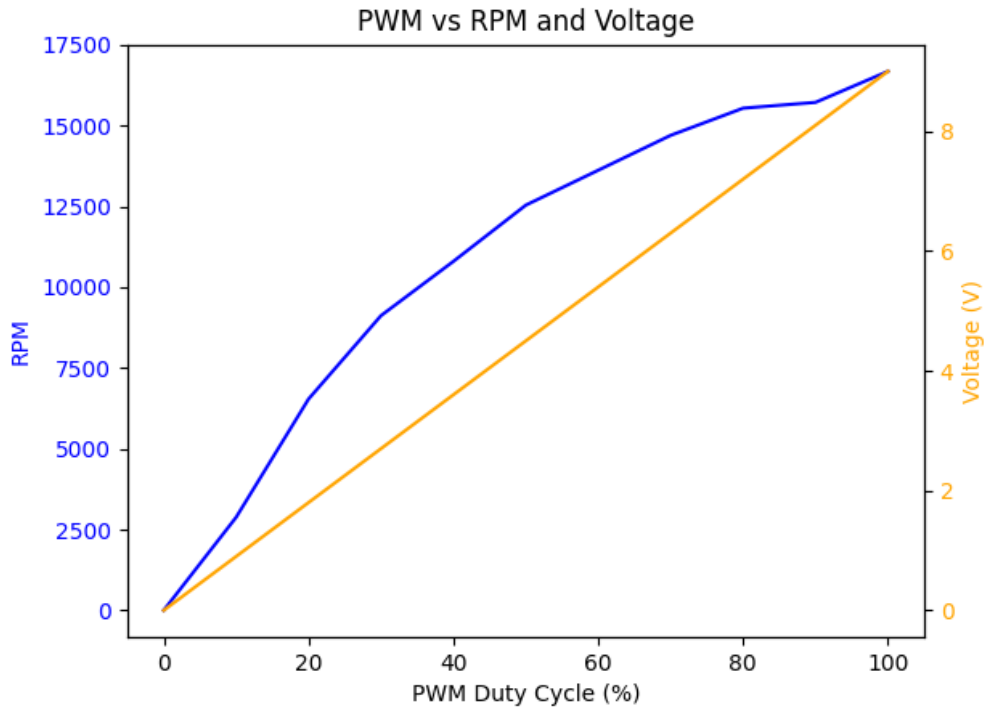


Figure 3: Comparison of motor output metrics (RPM and DC Voltage) vs PWM duty cycle

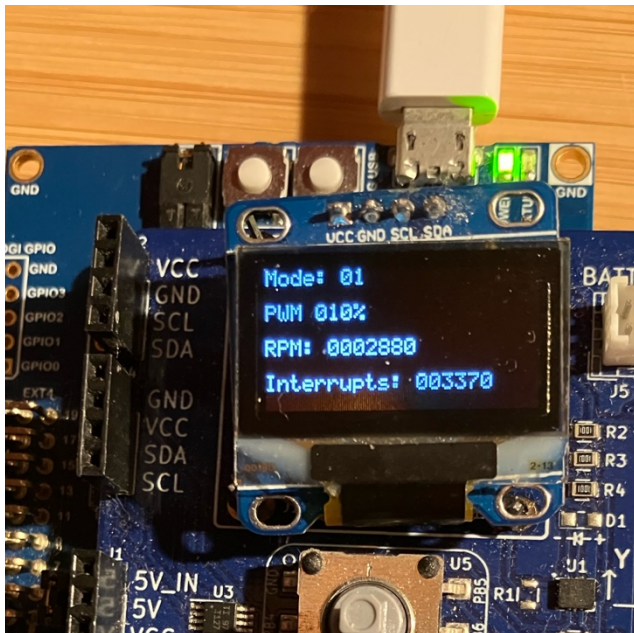


Figure 4: Sample RPM readings for 10 and 30% duty cycles. At 10% duty cycle the OLED shows Mode 1, PWM=10%, RPM=2880. At 30% duty cycle the OLED shows Mode 3, PWM=30%, RPM=11762. Both images confirm the numbers used in Table 2.

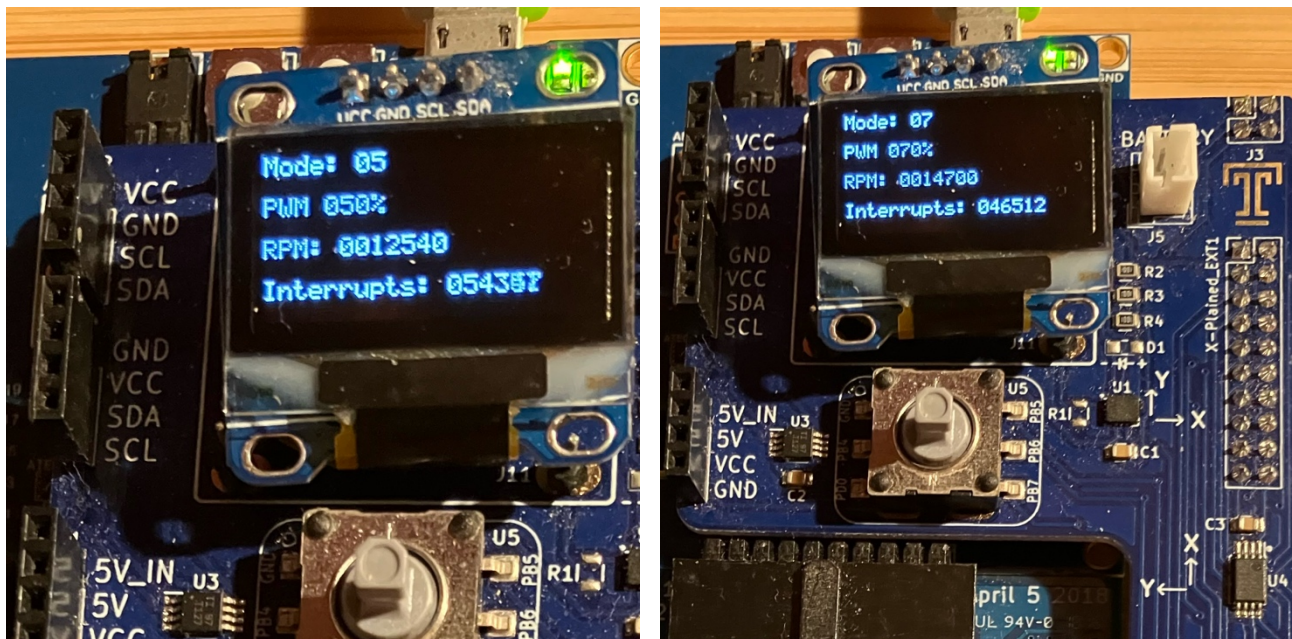


Figure 5: Sample RPM readings for 50 and 70% duty cycles. At 50% duty cycle the OLED shows Mode 5, PWM=50%, RPM=12540. At 70% duty cycle the OLED shows Mode 7, PWM=70%, RPM=14700. Both images confirm the numbers used in Table 2.

Explanation of result: The result shows a corresponding increase in motor speed as duty cycle rises. The RPM growth is roughly linear through the mid-range values, reflecting proportional control of average motor voltage. As the motor approaches its top speed, the spinning motor spins so fast that it induces its own voltage, lowering the current that can flow in. As a result, the increase becomes less linear at the higher duty cycles. The calculated DC voltage scales directly with the commanded duty cycle, aligning with the observed RPM trend. Note that graphs 2 and 3 are superimposed onto a single graph that displays DC voltage and RPM with respect to PWM % simultaneously (Figure 3).

While the RPM values seem high, they are consistent with the measured pulse rate. Manual tests confirmed correct sensor behavior: individual rotations consistently generated two interrupts, and a manual rotation of approximately one full revolution per second produce a consistent reading of RPM = 60. This validates that counting, timing, and RPM computation are functioning correctly.

IV. Discussion

a. Evaluation of Result 1

The average percent error between the calculated and measured ON-time (duty cycle) is 0.0174%, confirming that the PWM generation via Timer1 and associated ISRs accurately produces the intended duty cycles. Minor discrepancies are negligible and likely almost entirely due to imprecision placing the cursor markers in the Data Visualizer.

b. Evaluation of Result 2

The measured RPM values increase consistently as the PWM duty cycle rises, showing that the motor responds proportionally to the control signal. The relationship is approximately linear in the mid-range (10–70% duty cycle), though it plateaus at the higher duty cycles (80–100%). The results align with expectations for a DC motor under PWM: higher duty cycles produce higher average voltage and faster rotation. The top-end plateau may result from voltage generated by the spinning motor itself, which would reduce net voltage across the motor and thus current. Using linear scaling, expected RPM is calculated as:

$$RPM_{expected} = \frac{DutyCycle\%}{100} * RPM_{max}$$

For example, with an $RPM_{max} = 16680$, the 50% duty cycle yields an expected RPM of 8340. However, the nonlinear response of the motor at low voltages, due to factors such as static friction, produces significant errors. The average percent error is 44.38% across all ten points.

V. Conclusion

The project successfully implements a PWM-based speed control system for a DC motor using the ATmega324PB. Motor speed is adjusted through discrete duty-cycle levels, measured in real time using an opto-interrupter and external interrupts, and displayed on an I2C OLED. The combination of hardware timers for PWM, interrupt-driven pulse counting, and real-time display create a responsive and accurate closed-loop system demonstrating both motor control and speed monitoring. The PWM-generated duty cycles are experimentally verified according to theoretical expectations. The RPM is verified to scale with increased DC voltage in line with expectations.

VI. Appendix – Code with full comments

```
#define F_CPU 16000000
#include <avr/io.h>
#include <avr/interrupt.h>
#include "i2c.h"
#include "SSD1306.h"

#define up_count PB5 //pin corresponding to "up" position of RHC button
#define down_count PB7 //pin corresponding to "down" position of RHC button
#define DCmotor PE2
#define optointerrupter PD2
#define SLOTS 2 // number of slots per spinner revolution

double dc_adj=0.0; //duty cycle adjusting parameter
volatile char dutyCycle = 0; //global variable containing selected motor speed value
volatile char lastInput = 0; //variable to store the last state of PORTB
int count_int = 0; //set pwm value to 0 (no pwm) initially
volatile uint16_t opto_int = 0;
volatile uint16_t last_opto_initial = 0; // value at start of one second period for RPM
calc
volatile uint16_t rpm = 0; // pulses per sec -> RPM
int overflow_cnt = 0;

ISR(TIMER1_COMPA_vect){ //ISR for when timer1A interrupts
    PORTE &= ~(1<<DCmotor); //turn off PE2 pin
}
ISR(TIMER1_COMPB_vect){ //ISR for when timer1B interrupts
    PORTE |= (1<<DCmotor); //turn on PE2 pin
}
ISR(TIMER2_OVF_vect) {
    overflow_cnt++;
    if (overflow_cnt == 61) {
        rpm = ((opto_int - last_opto_initial) / SLOTS) * 60;
        last_opto_initial = opto_int;
        overflow_cnt = 0;
    }
}
ISR(INT0_vect) {
    opto_int++; // increment pulse counter on each slot
}

void OLED_display(){ //this will update the OLED display with the most current info
    OLED_SetCursor(0, 0);
    OLED_Printf("Mode: ");
    OLED_DisplayNumber(C_DECIMAL_U8,dutyCycle/10,2);
    OLED_SetCursor(2, 0);
    OLED_Printf("PWM%: ");
    OLED_DisplayNumber(C_DECIMAL_U8,dutyCycle,3);
    OLED_Printf("%% ");
    OLED_SetCursor(4, 0);
    OLED_Printf("RPM: ");
    OLED_DisplayNumber(C_DECIMAL_U8,rpm,7);
    OLED_SetCursor(6, 0);
    OLED_Printf("Interrupts: ");
    OLED_DisplayNumber(C_DECIMAL_U8,opto_int,6);
}
```

```

}

int main(void){
    OLED_Init(); //INITIALIZE THE OLED
    OLED_Clear(); //CLEAR THE DISPLAY
    DDRB = 0x00;    //set port b pins to inputs (for joystick)
    PORTB = 0xA0;    //set port b pins to have pull-up resistors
    DDRD &= ~(1 << optointerrupter); // set PD2 to input
    PORTD &= ~(1 << optointerrupter); // disables pullup
    DDRE |= (1<<DCmotor); //set motor pin to output

    // Configures external interrupt INT0
    EICRA |= (1 << ISC01); // ISC01=1 AND ISC00=0 = falling edge trigger
    EICRA &= ~(1 << ISC00);
    EIMSK |= (1 << INT0); // enable INT0

    // Timer initialization
    ICR1 = 65535; // full PWM period (4.1 ms), no prescale or preload -> (65536-
0)*1/16Mhz = 4.096 ms
    OCR1B=0;
    TCCR1A = (1<<COM1A0)|(1<<COM1B0); //enable timer1a and timer1b comparison
    TCCR1B = (1<<WGM13) | (1<<WGM12) | (1<<CS10); //enable timer1 with no pre-scaler
    TIMSK1 = (1<<OCIE1A)|(1<<OCIE1B); //enable timer1A and timer1B interrupts

    TCCR2A = 0; // Normal mode
    TCCR2B = (1<<CS22)|(1<<CS21)|(1<<CS20); // 1024 prescaler
    TIMSK2 = (1<<TOIE2); // enable overflow interrupt
    sei();

    while (1) { //infinite loop
        char joystick = (0b11110000&PINB); //get high 4 bits of PORTB (contains up and
down button)
        joystick = joystick^0xf0; //invert value of of up and down button (Pullup
resistors pull pins high)
        if(lastInput != joystick){ //check if PORTB has been updated
            if(joystick&(1<<up_count)){ //check for up button being pressed
                if(count_int < 10) //if the mode is less than 10,
                    count_int++; //YES: increment it
            }
            if(joystick&(1<<down_count)){ //check for down button being pressed
                if(count_int > 0) //if the mode is more than 0,
                    count_int--; //NO: decrement it
            }
            dc_adj=count_int * 0.1;
            dutyCycle = count_int*10; //update dutyCycle variable for OLED display
            lastInput = joystick; //keep the latest PORTB input
        }
        // Explicitly turn motor off if speed is zero
        if (dc_adj <= 0.0) {
            TIMSK1 &= ~((1<<OCIE1A)|(1<<OCIE1B)); //disable PWM interrupts
            PORTE &= ~(1<<DCmotor); //turn motor off
        } else {
            TIMSK1 |= (1<<OCIE1A)|(1<<OCIE1B); //enable PWM interrupts
            OCR1A = (uint16_t)(ICR1 * dc_adj); //reassign timer1 value for On-time pulse
        }
        OLED_display(); //update OLED display
    }
}

```